

Informe Tarea 1

Pablo Guajardo Vásquez, pguajardo@usm.cl

Gastón Urrutia Fuentes, gurrutia@usm.cl

Vicente Emperanza Pérez, vemperanza@usm.cl

Ignacio Beltrán Lara, ibeltran@usm.cl

Agustín Henríquez Barros, ahenriquezb@usm.cl

1. Parte 1

1.1. Diseño de la función lecturaSensor()

En la Figura 1 se muestra el diagrama de flujo de la función **lecturaSensor()**. La función consiste en la simulación de la lectura de un sensor, el cual entrega un valor (número entero) entre 0 y 255 que es guardado en una variable (**lecturaAleatoria**) y retornada, para esto usamos la función **rand()** % 256, la cual nos entregará cada vez un número aleatorio entre el rango solicitado, notemos que para que esto funcione es necesario incluir la librería **<stdlib.h>**.

Además, hacemos uso del enum **Accion**, el cual mejora la legibilidad y el orden del código, ya que nos permite poner nombres a lo que normalmente expresamos como constantes, volviendo no solamente más intuitivo el entendimiento del código, sino que también ayudando a recordar el funcionamiento de cada parte en caso de corregir errores o realizar cambios.

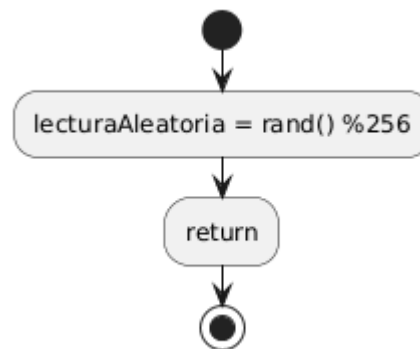


Figura 1

1.2. Diseño completo de la nueva función main()

En la Figura 2 se muestra el diagrama de flujo de la función **main()**. La nueva función usa el enum **Accion** declarado anteriormente para aplicar los diferentes casos del estado de los sensores (INIT, READ, SHOW), cada caso llama a la función **usarSensores()**, la cual revisa el estado del sensor mediante las variables **estadoInit** y **estadoRead** y devuelve el mensaje cuando correspondiente (en el caso de READ Y SHOW).

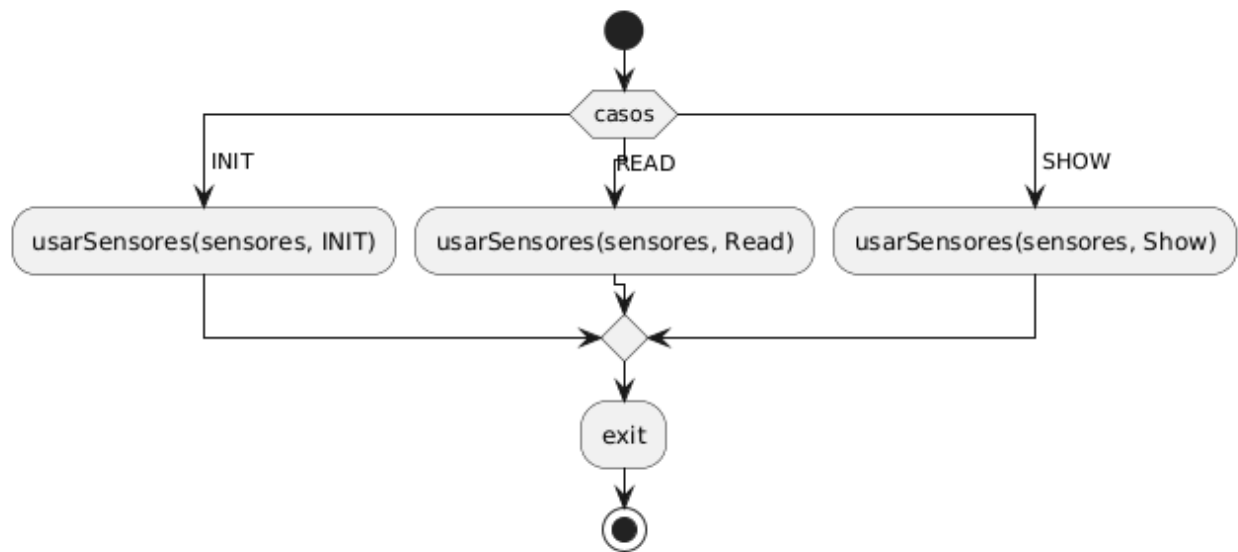


Figura 2

2. Parte 2

2.1. Hito 1

- 1.- La función `clock()` retorna el tiempo que ha pasado desde la ejecución del código, la cual esta medida en la cantidad de **tics** que ha realizado el **procesador**.
- 2.- Las variables `inicio` y `fin` utilizan la unidad [tic].
- 3.- Porque con la diferencia se obtiene el Δt (delta t) que corresponde a la duración del ciclo for, luego se divide por la constante `CLOCKS_PER_SEC` (que es definida por cada sistema operativo) para hacer el cambio de unidades de tic a segundos (`CLOCKS_PER_SEC` tiene unidad [tic/segundos]).
- 4.- Convierte explícitamente la resta (`fin-inicio`) en un tipo de dato `double`, el cual almacena un numero flotante de 8 Bytes.
- 5.- El uso de `double` es necesario para que el resultado de la división sea preciso y entregue un número `double` en vez de un entero, en caso contrario el número entero se vería truncado, lo que disminuiría la precisión.

2.2. Hito 2

En la Figura 3 se muestra el diagrama de flujo de la función **cicloPrueba()**. La función toma el caso pedido según el enum `caso_t` (VACIO, ESPACIO, ASTERISCO, NEWLINE) e inicia la función **clock()**, la cual se guarda en la variable **inicio**, luego crea una variable local **t** y la compara con la variable **cantidadIteraciones** (definida fuera de los ciclos), una vez que **t** sea igual que la variable se detiene la función `clock()` y se guarda el dato en la variable

fin, finalmente se calcula el promedio de las mediciones y se guarda en la función **tiempoPromedio**.

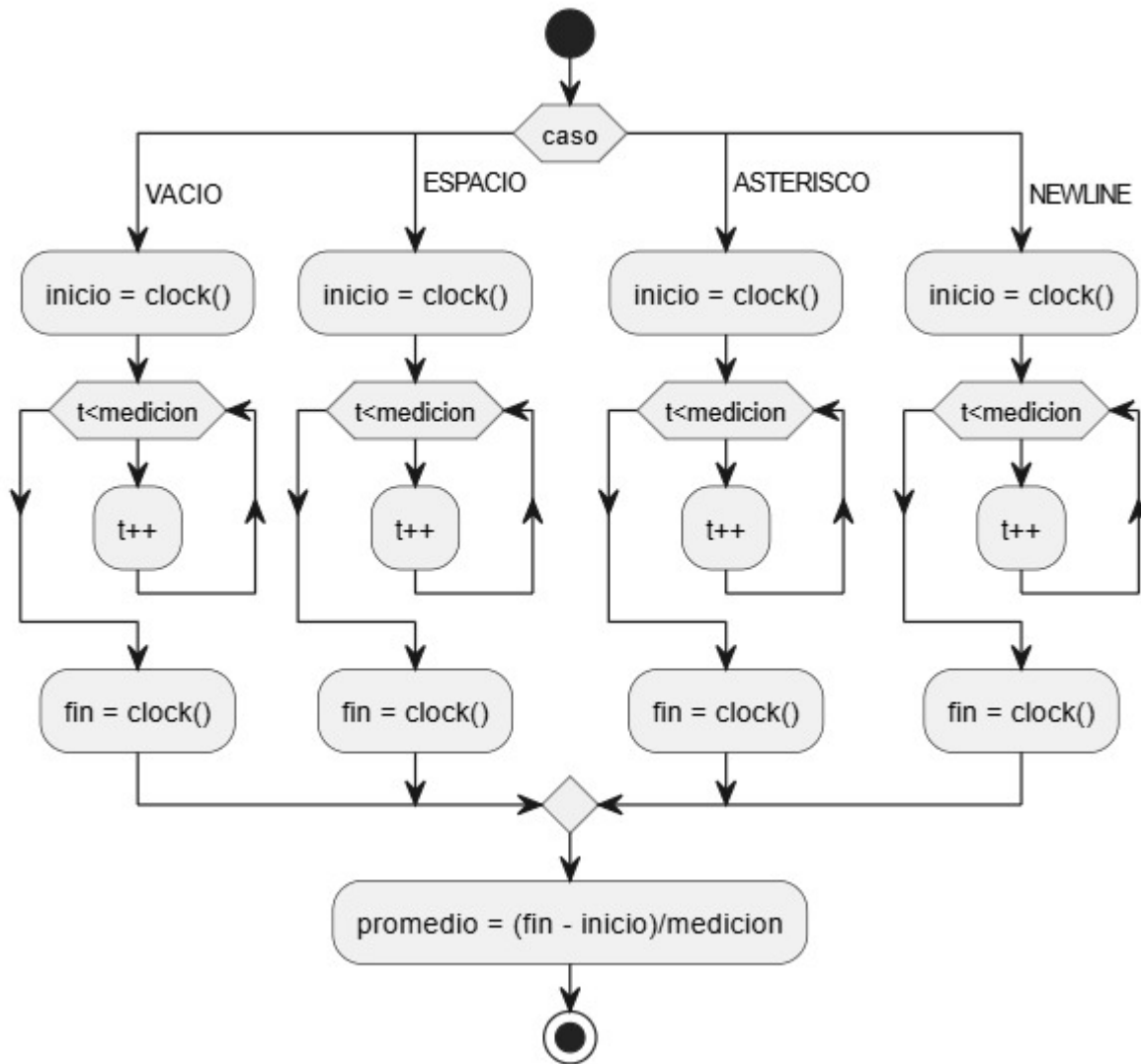


Figura 3

2.3. Hito 3

Para la medición de los tiempos de ejecución primero se toman las variables **fin** e **inicio** y se restan (**fin-inicio**), lo que nos da un Δt (tiempo transcurrido en el intervalo), luego se divide el resultado por **CLOCK_PER_SEC** (cantidad de tics por segundo devuelto por la función **clock()**) y se multiplica por $1e9$, todo esto se guarda en la variable **tiempoTotal**, la cual luego es dividida por **cantidadIteraciones** para finalmente obtener el **tiempoPromedio**. Se midieron $1e8$ mediciones (variable **cantidadIteraciones**).

Características computador (MacBook air m2):

Chip M2 Apple

CPU de 8 núcleos (4 rendimiento y 4 eficiencia)

GPU de 8 núcleos

Neural Engine de 16 núcleos

100 GB/s de ancho de banda de memoria

RAM 8 GB memoria unificada

Versión compilador: Apple clang version 21.0.0 (clang-2100.0.123.102)

Target: arm64-apple-darwin25.4.0

Thread model: posix

En la Tabla 1 se muestran los datos obtenidos de las mediciones de tiempo de ejecución. Como podemos observar en la desviación estándar, en los casos NADA, ESPACIO Y ASTERISCO las mediciones varían muy poco con respecto al promedio, en especial el caso NADA que tiene la menor desviación estándar, esto significa que poseen cierto grado de consistencia en sus medidas, en cambio el caso NEWLINE posee una alta desviación estándar, lo que significa que sus mediciones se alejan más del promedio, en otras palabras, sus mediciones son menos consistentes comparadas con los demás casos.

Caso	tprom 1	tprom2	tprom3	tprom4	tprom5	tprom6	tprom7	tprom8	tprom9	tprom10	Promedio	Desv. Estandar
NADA	1,217900	1,047360	1,043220	0,959270	0,913200	1,042130	1,204140	1,024090	1,031250	1,032340	1,05149	0,08962319
ESPACIO	44,54158	44,16544	44,09261	43,84943	44,56945	44,64666	44,46969	44,40642	44,11860	44,98090	44,384078	0,313245474
ASTERISCO	44,21352	43,86769	44,15177	43,89512	44,43702	44,39618	44,29008	44,76754	44,15745	45,14683	44,33232	0,367965152
NEWLINE	683,02457	697,17435	655,03838	679,75398	673,97785	681,74014	687,87286	679,91075	676,81498	682,39913	679,770699	10,21718149

Tabla 1